From Tools to Missions: A Mission-Capability Runtime for Enterprise AI

Credible Lineage, Measurable Post-LLM Capability Scoping, Engineering Proof, and a Prerequisite-Conditioned Long-Horizon Roadmap

Prasad Deshpande
Independent Researcher — Agentic AI Systems and Enterprise Architecture
ORCID: 0009-0008-1344-6585
Email: er.prasad.deshpande@gmail.com
Research & Architecture: https://prasaddeshpande.com/

MCR R1.9 External-Review Candidate R3 (release 1.9.0) — 20 October 2026
Source of record: Adversarially Hardened Whitepaper Candidate R1.6.1 — 31 August 2026.

Publication-layer provenance. This draft extends the original adversarially hardened MCR R1.6.1 whitepaper without changing the frozen MCWS experiment. The original scientific manuscript is anchored by SHA-256 `E589393C6EEA0BA92E4EC521D1577A6F561F9ADFA38FF9EEE97A94695C666D90`. R1.7, R1.8 and R1.9 contribute separately frozen runtime and enterprise-engineering evidence. No historical failure is rewritten as success. In particular, the R1.9B live SAP mutation study did not reach validated completion: CREATE reached the provider, ACTIVATE did not, and the authoritative repository state after CREATE remains unresolved at this draft. Evidence-portability limits are stated openly: the bytes of the frozen scientific manuscript itself are not available in the repository (only its hash and the publication layer derived from it), and six signed historical scripts survive only as normalized reading copies (Appendix F).

Abstract

Goal-driven service discovery, semantic capability descriptions, durable workflow state, compensation and dynamic provider binding have substantial ancestry in Semantic Web Services, SOA/BPM and durable-workflow systems. LLM-era tool-retrieval work likewise predates MCR and already recognizes the model-visible tool set as part of the reasoning surface. The narrower problem addressed by the Mission-Capability Runtime (MCR) is how an enterprise runtime can make executable capability exposure mission-stage-specific while keeping authorization, final provider ranking, mutation authority and binding outside probabilistic model control.

The frozen MCWS scientific result from R1.6.1 is unchanged: MCWS achieved 30/48 primary mission successes (62.5%), Static 17/48 (35.4%) and Broad 5/48 (10.4%). Holm-adjusted case-cluster tests were `p = 0.00293` for Broad–MCWS, `p = 0.03125` for Static–MCWS and `p = 0.2266` for Broad–Static. These results evaluate capability-exposure behavior in the frozen benchmark; they do not establish general enterprise safety, universal orchestration superiority, production cost savings or transaction correctness.

R1.7 and R1.8 then moved the research program from architecture synthesis toward bounded runtime evidence. They exercised deterministic authority boundaries, provider contracts, multi-provider capability surfaces, point-of-use contract revalidation, mission-scoped capability identities, fail-closed drift detection and bounded multi-mission execution. R1.9A formalized governed execution using PrincipalContext, DelegationEnvelope, MissionExecutionEnvelope, MutationBudget, TransactionEnvelope, ExecutionReceipt, OutcomeReceipt, CompensationGrant, ContractLease, MissionCheckpoint and ContinuationLease, while keeping the authorization fast path deterministic and off the LLM critical path.

R1.9B prospectively froze one live SAP mutation mission: target `TRL_EN`, object `ZCL_MCR19B_9533C4D8`, package `ZCLINE`, provider ADT, exactly two authorized mutations (CREATE and ACTIVATE), two outcome-validation operations, zero retries and no compensation or cleanup authority. Successive fail-closed iterations exposed real implementation defects in authorization binding, state-root handling, evidence persistence and MCP session semantics. The final qualified HF06 runner reached the CREATE call on the ADT MCP provider (reach of the SAP back end is not established) and stopped on `CREATE_FILEPATH_MISSING` before ACTIVATE. A single-use lock prevents replay. Subsequent read-only/validation-only reconciliation could not establish whether CREATE persisted in SAP (the retained validation response is a provider error stating that ADT discovery could not be initialized, so it carries no information about the object), so `CREATE_COMMIT_STATE=UNKNOWN` remains the bounded closure state.

The R1.9 result is therefore deliberately partial. Around that live attempt it exercised deterministic control-plane properties: source-bound authorization, session-aware MCP execution, a frozen two-mutation budget, a single-use lock written before the first side effect, and a fail-closed stop with no retry. The lock's refusal of a replay was not itself exercised, and the budget was not consumed to completion. It does **not** establish successful end-to-end live mutation, successful activation, authoritative outcome completion, autonomous self-healing, general transaction atomicity or production enterprise safety. The partial result sharpens rather than weakens the central runtime distinction: a tool request, a provider response, a committed side effect, an execution receipt and a validated enterprise outcome are different states.

> **REQUESTED != AUTHORIZED != BOUND != EXECUTED != COMMITTED != VALIDATED_COMPLETE**
>
> **The LLM proposes semantics; the runtime binds authority; enterprise state decides completion.**

Keywords: agentic AI; enterprise agents; Mission-Capability Runtime; Minimum Capability Working Set; MCP; tool retrieval; capability scoping; deterministic authority; mutation budgets; execution receipts; outcome validation; SAP; enterprise architecture

---

1 Executive summary

The original MCR whitepaper argued for a narrow layer above provider protocols and enterprise gateways: durable mission state should produce a current capability requirement; only a bounded stage working set should become model-visible; the LLM may propose semantic intent; deterministic runtime logic must own admissibility, provider ranking, binding integrity and execution authority; and authoritative enterprise observation should determine whether the mission actually progressed.

R1.9 preserves that thesis but changes the evidence balance. The program now contains five contribution levels, drawn from seven evidence streams (Appendix D):

Contribution level	R1.9 position	Evidence class	Explicit boundary
Scientific	Stage-scoped MCWS Broad/Static/MCWS benchmark	Frozen experiment	Does not establish universal MCR superiority or production safety
Engineering	SAP/MCP/Capella/BAS plus R1.7/R1.8 provider/runtime proofs	Separate frozen POCs	Components were exercised separately; not one monolithic production runtime
Runtime-control research	R1.9A deterministic governed-execution foundation; R1.9B bounded live mutation attempt	Synthetic/local + bounded live evidence	Full mutation-to-authoritative-outcome completion not established

Contribution level	R1.9 position	Evidence class	Explicit boundary
Architecture synthesis	Mission -> MCWS -> deterministic admissibility/binding -> execution -> authoritative outcome	Practitioner pattern	Components have prior-art ancestry; combination and control discipline are the contribution
Long-horizon agenda	portable missions, trust federation, economics, cyber-physical execution	Hypothesis / scenario	Prerequisite-conditioned and non-empirical

The practical MCR rule remains:

> **The LLM proposes semantics; the runtime binds authority; enterprise state decides completion.**

The R1.9 outcome rule becomes equally important:

> **REQUESTED != AUTHORIZED != BOUND != EXECUTED != COMMITTED != VALIDATED_COMPLETE**

A runtime that collapses those states cannot safely reason about partial failure.

MCR does not claim to have invented service discovery, durable workflows, Saga compensation, dynamic service binding or tool retrieval (Section 16). Its residual contribution is the disciplined combination of mission-scoped capability selection, deterministic execution authority, provider/tool binding, point-of-use revalidation, mutation budgets, transaction state, receipts, an explicit outcome distinction, and continuation/replay control at the enterprise-agent execution boundary.

1.1 Decision rule

Prefer a static/compiled execution path where provider identity, mission requirements and authority are stable. Introduce MCR where one or more of the following are material:

- provider substitution or failover;
- changing mission requirements;
- heterogeneous capability surfaces;
- delegated authority;
- long-running state;
- consequential mutations;
- contract/schema drift;
- point-of-use policy changes; or
- authoritative outcome verification.

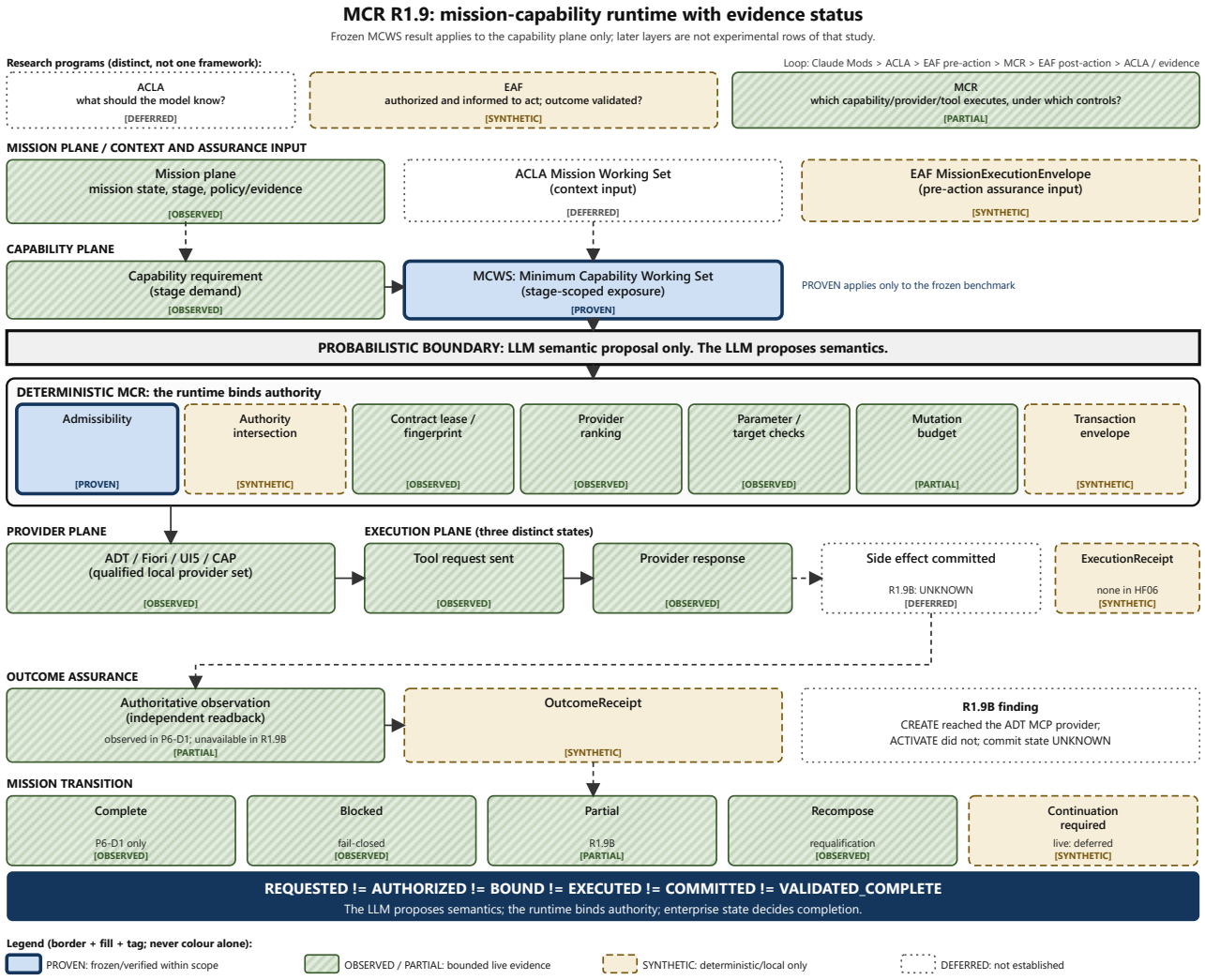
1.2 Six states that must not be collapsed

State	Meaning	R1.9B / HF06 status
REQUESTED	The agent or runner issued a tool request	CREATE request issued once
AUTHORIZED	Deterministic authority admitted that request	Exact two-mutation authorization existed; not reusable
BOUND	A concrete provider/tool/target/parameter set was bound	ADT, `TRL_EN`, `ZCL_MCR19B_9533C4D8`, `ZCLINE`

State	Meaning	R1.9B / HF06 status
EXECUTED	The provider received and answered the call	CREATE reached the ADT MCP provider and a parseable response came back (SAP back-end reach not established); ACTIVATE did not reach the provider
COMMITTED	The side effect persisted in the system of record	**UNKNOWN**
VALIDATED_COMPLETE	Authoritative observation matched the intended outcome	**Not reached**

An execution receipt is a record of invocation evidence. It is not a record that state changed, and it is not a record that the mission is complete. In HF06 no execution receipt was written at all, because the client stopped when the CREATE response lacked the expected file path; the ambiguity was recovered afterwards only by separate read-only reconciliation.

Figure 1. MCR R1.9 architecture with evidence status. Each element is tagged PROVEN, OBSERVED, PARTIAL, SYNTHETIC or DEFERRED by border style, fill pattern and text. The frozen MCWS result applies only to the capability plane. Source: `architecture/MCR_R1.9_ARCHITECTURE_FIGURE_R3.svg`; specification: `architecture/MCR_R1.9_FINAL_ARCHITECTURE_FIGURE_SPEC_R3.md`.



1.3 Claim boundary

R1.9 is not a claim that MCR has become a production transaction coordinator. It is a bounded research program that progressively tests the control structures required before such a claim could be credible.

2 What we have already built: engineering proof at a glance

The evidence program remains split into independent streams.

2.1 Original scientific stream

The frozen Broad/Static/MCWS benchmark remains unchanged.

Corrected primary mission success:

Architecture	Success	Rate
Broad	5/48	10.4%
Static	17/48	35.4%
MCWS	30/48	62.5%

The result supports bounded stage-scoped capability exposure in the evaluated benchmark. It does not independently establish attack-surface reduction, monetary savings, zero-trust isolation or general security.

2.2 R1.7 governed-runtime stream

R1.7 added bounded live runtime evidence:

- read-only provider binding and dry-run paths;
- bounded live SAP reads;
- actual provider-loss fail-closed behavior;
- explicit evidence that autonomous ADT resurrection was **not observed**;
- federated authority distinctions;
- outcome-authoritative state handling; and
- one separate bounded SAP write experiment (P6-D1, object `ZCL_MCR_P6D_DFC3F677`, package `ZCLINE`, target `TRL_EN`): exactly two authorized mutations, two consumed, zero unauthorized or additional writes, an initial ATC result timeout after which no write was replayed, and `VALIDATED_COMPLETE` only after independent read-only ATC observation (0 findings).

The P6-D1 object is a different object from the R1.9B object. It is historical R1.7 evidence and is **not** an R1.9 result and is not reused as one.

R1.7 is a separate frozen stream from the MCWS benchmark. It rests on the companion R2-E series (P3–P7): controlled and adversarial admission comparisons, a bounded live read-only enterprise proof of concept (6/6 eligible live reads, 0 writes), temporal/fault and federated-authority tests, and one retained negative result: after actual MCP server-process termination, automatic ADT process resurrection was not observed and explicit host recovery was required.

The surviving invariants include:

- `DISCOVERABLE != AUTHORIZED`
- `TRUSTED_TO_DESCRIBE != AUTHORIZED_TO_EXECUTE`
- delegated authority cannot amplify;
- provider equivalence is directional rather than universal;
- `EXECUTED != COMPLETE`.

2.3 R1.8 multi-provider / multi-mission stream

R1.8 qualified a four-provider local capability set across ADT, Fiori, UI5 and CAP and formalized provider-qualified capability identities using:

`PROVIDER::TOOL_NAME`

The local qualified set contained 41 tool contracts, with 30 classified as zero-write READ/ANALYSIS capabilities and the remainder separated into local or enterprise mutation classes.

R1.8 live work demonstrated:

- one read-only call per provider in the C1 bounded qualification;
- mission-aware capability identities;
- fail-closed contract mismatch detection;
- Fiori contract drift despite unchanged nominal package/server version;
- explicit requalification before continuation;
- bounded multi-mission continuation after governed repair; and
- no automatic replay of failed business calls.

The observed Fiori drift supports a central contract lesson:

> **VERSION_EQUALITY_DOES_NOT_IMPLY_CONTRACT_EQUALITY**

Scope of that observation: the Fiori contract drift was observed relative to the C1 freeze. The raw historical C2B Fiori contract was not captured, so the structural attribution of the earlier drift is not independently proven from raw capture. The C2 closure covers three validated missions, eight unique executed business calls, eight valid outcomes, zero retries, zero replay and zero writes, through a governed recovery lifecycle. It does not claim uninterrupted multi-mission replication, autonomous recovery, universal provider equivalence or universal drift detection.

2.4 R1.9A governed-execution foundation

R1.9A is complete and frozen.

It defines:

- `PrincipalContext`
- `DelegationEnvelope`
- `MissionExecutionEnvelope`
- `MutationBudget`
- `TransactionEnvelope`
- `ExecutionReceipt`
- `OutcomeReceipt`
- `CompensationGrant`
- `ContractLease`
- mission-relevant contract hash/change classification
- `MissionCheckpoint`
- `ContinuationLease`

The deterministic fast path is designed so that:

- LLM authorization calls are not required;

- a remote governance network hop is not mandatory per invocation;
- authority amplification is refused;
- unauthorized compensation is refused;
- point-of-use checks remain deterministic.

Evidence class: SYNTHETIC / LOCAL. The R1.9A test report records PASS for schema validation, canonical serialization, deterministic hashing, idempotent compilation, authority attenuation, budget exhaustion, expiry handling, contract-lease handling, contract-change classification, checkpoint integrity, continuation exclusion, compensation separation and fast-path determinism, with zero determinism drift. A local microbenchmark (5,000 iterations, no network, MCP, business or model calls) measured a simulated governed admission path at about 0.28 ms against about 0.0012 ms for a minimal direct-dispatch simulation. Its p50, p95 and p99 values are identical, so it reports a single aggregate figure, not a latency distribution; it measures local governance logic only and says nothing about enterprise end-to-end overhead.

---

3 Independent-review alignment: what changed

The original adversarial review narrowed three important areas:

1. provider abstraction belongs to the deterministic MCR binding layer, not MCWS;
2. the MCWS benchmark measures selection/task-success behavior, not end-to-end security; and
3. composite substitution under Saga-style compensation remained incompletely specified.

R1.7–R1.9 do not reverse those concessions.

Instead, the program tests the runtime boundary directly and exposes new failure classes:

- authorization artifact can be semantically correct but bound to the wrong signed source;
- a runner may qualify local request construction yet fail on the real authorization path;
- evidence persistence code can fail after a real mutation unless pre-mutation replay controls exist;
- MCP initialize success does not imply subsequent requests are valid unless session semantics are preserved;
- localized DateTime conversion can invalidate otherwise correct signed authority;
- a client can fail to interpret a provider response after the mutation request has already been sent;
- after that boundary, replay is unsafe because the side effect may already have committed.

These are not paper-cleanup details. They are exactly the runtime states a mission system must distinguish.

---

4 Where the runtime fits

4.1 Protocol, control plane and mission runtime are different layers

MCR does not replace:

- MCP;
- A2A;
- API gateways;
- registries;
- OAuth/workload identity;
- enterprise authorization;
- SAP business authorization;

- observability; or
- systems of record.

It consumes those layers.

A useful separation is:

```
Mission semantics -> MCR -> enterprise control plane -> protocol adapters -> systems of record
```

4.2 Mission state, not provider identity

The core indirection remains:

```
Mission -> Capability -> Provider -> Tool
```

The mission should remain stable when a provider changes, but provider change is not automatically safe. Substitutability must be directional, stage-relative, policy-compatible and authority-preserving.

4.3 Relationship to ACLA and EAF

The broader research program separates three questions:

- **ACLA:** what should the model know?
- **EAF:** is the action sufficiently supported and authorized, and was the outcome validated?
- **MCR:** which capability/provider/tool should execute the authorized mission under deterministic runtime constraints?

MCR consumes context and authority; it should not silently become the owner of all context lifecycle or all enterprise assurance.

---

5 The measurable post-LLM problem: capability catalogs become reasoning input

The original post-LLM observation remains unchanged.

Classical registries are external deterministic data structures until queried. In LLM systems, tool/capability descriptions can be serialized directly into the active model context. A broader executable catalog can therefore alter:

- prompt length;
- semantic collisions;
- model tool selection;
- exposure to malicious descriptions;
- audit clarity; and
- reasoning ambiguity.

MCWS addresses the model-visible capability surface.

MCR addresses deterministic admissibility and execution after the model proposes semantic intent.

The two mechanisms must not be conflated.

---

6 Practical implementation profile

6.1 Effective execution authority

R1.8/R1.9 formalize effective authority as an intersection rather than an inherited model privilege:

```
EffectiveExecutionAuthority = ProviderAuthority ∩ UserAuthority ∩ AgentAuthority ∩ MissionAuthority ∩ TemporalAuthority ∩ CapabilityAuthority ∩ TargetAuthority ∩ TransactionAuthority
```

Any empty term blocks execution.

6.2 Minimum Capability Working Set

MCWS is the model-visible capability projection for the active mission stage.

Its job is to bound the reasoning surface. It is not final permission.

6.3 Deterministic binding

A model proposal is an untrusted semantic candidate.

Before execution, MCR should revalidate at least:

- mission stage;
- capability class;
- provider identity;
- contract fingerprint;
- target;
- parameter digest;
- authority chain;
- transaction state;
- mutation budget;
- temporal validity; and
- current policy.

6.4 Runtime records

The research lineage uses runtime records rather than proposing a new industry standard.

R1.9 operational objects include:

- MissionExecutionEnvelope
- Runtime Binding Record or equivalent binding receipt
- MutationBudget
- TransactionEnvelope
- ExecutionReceipt
- OutcomeReceipt
- ContractLease
- MissionCheckpoint / ContinuationLease
- CompensationGrant

The important property is explicit state, not the exact serialization vocabulary.

6.5 Contract leases and drift

A nominal version string is insufficient.

MCR maintains:

- full provider contract identity; and
- mission-relevant contract identity.

Point-of-use drift may therefore invalidate only the affected mission path rather than globally declaring a provider unusable.

6.6 Outcome verification

For consequential operations:

```
tool returned success != enterprise outcome validated
```

An execution receipt records invocation/execution evidence.

An outcome receipt requires authoritative observation against the intended state/delta.

---

7 SAP S/4HANA worked example and R1.9B live study

The original R1.6.1 paper used SAP as a worked enterprise example while keeping that example separate from the frozen MCWS benchmark.

R1.9B moves one narrow SAP mutation into live bounded research.

7.1 Frozen live mission

The prospective live contract fixed:

- Target: `TRL_EN`
- Object: `ZCL_MCR19B_9533C4D8`
- Package: `ZCLINE`
- Provider: ADT
- Mutation budget: exactly `2`
- Mutation 1: `abap_creation-create_object`
- Mutation 2: `abap_activate_objects`
- Validation 1: `abap_atc_run`
- Validation 2: `abap_atc_get_result`
- Automatic retries: `0`
- Compensation authority: `FALSE`
- Cleanup/delete authority: `FALSE`
- Substitution: `FALSE`

The object, target, package, provider and budget were not allowed to change to rescue the study.

7.2 Qualification chronology

The live runner lineage intentionally preserves failed attempts as evidence.

Historical defects included:

- initial artifact/source-map mismatch;
- auth-context property mismatch;
- qualification guard blocking live execution;
- undefined StateRoot;
- stale historical authorization binding;
- undefined evidence writer;
- missing pre-mutation single-use lock;
- missing MCP session ID propagation;
- missing `notifications/initialized`;
- launcher-only DateTimeOffset parsing defect.

No failed historical runner is reusable.

Evidence note: this chronology of earlier runners (R1, HF01–HF05) is owner-reported. Their transcripts are not retained in this repository; only HF06 attempt 2 has retained output. The defects are reported as lessons, not as independently re-verifiable measurements.

7.3 HF06 qualified state

HF06 became the first runner to pass the full pre-write control set:

- source/signature binding;
- unchanged authorization validator;
- frozen scientific contract;
- HF06-only authorization path;
- session ID capture/propagation;
- `notifications/initialized`;
- tools/list before mutation lock;
- single-use lock before CREATE;
- signed mock E2E;
- pre-provider construction checks;
- exact signed session-protocol qualification.

Frozen HF06 source:

`DE183ABC47B8F025ED60CFE5CFEA4527585FB20154C949628BDE95E42A0D3C88`

Evidence note: the pre-write qualification steps listed above (including the signed mock end-to-end and the session-protocol qualification) are reported by the run record; their outputs are not retained here, and the original signed bytes of this source are not in the repository (only a normalized reading copy whose line 195 matches the line quoted in the retained error output). The frozen hash is corroborated by the single-use lock's `script_sha256`.

7.4 HF06 live attempt 2

The final live attempt reached:

`CREATE -> response -> filePath extraction -> CREATE_FILEPATH_MISSING -> STOP`

Observed:

- CREATE provider call reached;
- ACTIVATE provider call did not reach;

- one single-use lock persisted;
- no CREATE execution receipt was written;
- no ACTIVATE execution receipt was written;
- no outcome receipt was written;
- no final receipt was written;
- authorization-consumed receipt was not written;
- automatic retry remained zero.

Cause boundary: the runner's code path continues to the file-path check after any non-empty, parseable response, and it does not inspect a JSON-RPC `error` member first. The missing file path is therefore consistent with a success payload in a different layout and equally with an error payload that did not commit. The raw CREATE response was not preserved, so the cause cannot be determined from retained evidence.

The single-use lock prevents replay of the same authority.

7.5 Unresolved commit state

The central unresolved question is whether the CREATE side effect committed before client-side response interpretation failed.

Accounting note: the reconciliation adjudication record reports `prior_sap_writes: 0`. That figure describes the reconciliation run only. It must not be read as a statement that HF06 attempt 2 performed zero SAP writes; the CREATE call reached the provider and may have committed.

A direct authoritative repository readback was attempted through a previously used Eclipse bridge, but the bridge was unavailable.

A secondary creation-preflight reconciliation used bounded read/validation MCP operations and captured responses, but local adjudication did not yield a reliable existence-collision signal or reliable positive-validation signal. The retained validation response is an error (`isError: true`: "Unable to initialize the ADT Discovery. Check the availability of the back-end system or restart the IDE."); the details response returned only generic creation-field metadata. Neither says anything about whether the object exists, so the absence of a signal is not evidence of absence. The same class of infrastructure error is one possible explanation for the CREATE response lacking a file path; this is a possibility, not a finding.

Current state:

`CREATE_COMMIT_STATE=UNKNOWN`

`OBJECT_STATE_INFERENCE=UNKNOWN`

`AUTHORITATIVE_REPOSITORY_READBACK=FALSE`

This state must not be rewritten as either success or absence without stronger evidence.

7.6 Why HF06 is scientifically valuable despite incomplete completion

HF06 is not a failed demonstration to be repaired by another run. It is a bounded partial execution result that exposes ambiguous post-side-effect state, which is the exact condition a mission runtime must handle:

1. Control-plane properties held through the live boundary: exact-source and exact-authorization binding, session-aware MCP sequencing, a single-use lock created before the first side effect, and a budget frozen at two mutations.
2. The mission stopped at the client's own check that the CREATE response contained a file path, after the request had left the client. Whether that response was an error or a success in an unexpected layout is

undetermined, which is precisely the case where retry and replay are unsafe.
3. The runtime stopped. It did not retry, did not proceed to ACTIVATE on an unverified premise, and did not delete, compensate or substitute.
4. The result forced an honest state vocabulary: provider reach, commit and validated completion had to be separated, and commit remained `UNKNOWN` because authoritative readback was unavailable.

What is not shown: successful end-to-end completion, successful activation, authoritative outcome validation, autonomous recovery, or that CREATE did or did not persist.

7.7 Closure decision

No further SAP mutation is authorized. HF06 must not be rerun and its authorization must not be reused. Generating a new mutation to resolve the unknown would convert a defensible record into a cleaner but different experiment. The unknown is retained as the result.

---

8 Adoption roadmap: from broad exposure to governed mission execution

The original incremental roadmap remains useful, with R1.9 evidence added:

1. **Broad exposure** — inventory tools and establish baseline.
2. **Governed catalog** — registry/gateway/identity/policy.
3. **Capability normalization** — provider-neutral capability IDs.
4. **Stage MCWS** — expose only current mission capability set.
5. **Runtime recomposition** — directional provider qualification and point-of-use revalidation.
6. **Consequential execution** — delegated authority, mutation budgets, transaction state and replay protection.
7. **Authoritative outcome** — independent readback and outcome receipt.

R1.9B illustrates why stages 6 and 7 cannot be collapsed.

---

9 Identity and policy: how much control is enough?

MCR treats business authorization as external authoritative input, not a model inference.

Relevant runtime principles:

- discoverability does not grant authority;
- trusted capability descriptions do not grant execution authority;
- delegated authority cannot exceed parent authority;
- stale authority must fail closed;
- mutation budgets are explicit;
- compensation authority is separate;
- authorization must bind exact mission/source/target semantics; and
- point-of-use revalidation is required where material state can change.

---

10 Engineering failure modes observed in practice

The MCR program now contains an unusually useful failure register.

Failure class	Observed form	Runtime lesson
Provider loss	ADT process loss in earlier bounded study	Current provider health is mission state; autonomous self-healing was not observed
Contract drift	Fiori normalized contract changed under nominally unchanged package/server version	Version equality does not prove contract equality
Authorization binding	Correct authorization data pointed at stale runner identity	Authority must bind exact executable source
Evidence persistence	Undefined evidence writer discovered in live-path audit	Post-mutation evidence failure must not permit replay
Single-use semantics	Missing pre-mutation lock in HF04	Acquire replay barrier before first side effect
MCP session semantics	initialize succeeded but later request lacked session ID	Protocol session state is part of execution state
Initialization lifecycle	`notifications/initialized` omitted	MCP lifecycle sequence must be explicit
Time parsing	ISO expiry converted through localized DateTime string	Authority time must use unambiguous round-trip representation
Response contract mismatch	A CREATE response was returned and parsed, but the expected `filePath` was absent; the client did not check for a JSON-RPC error first, and the raw response was not preserved	Provider response, provider success and client interpretation are different states; error payload versus alternate success layout cannot be distinguished from preserved evidence
Outcome uncertainty	CREATE may have committed; ACTIVATE not reached; readback unavailable	Partial execution must remain explicit until authoritative reconciliation
Evidence-artifact fidelity	Git line-ending normalization altered hash-bound, signed evidence files in the first portability commit	Hash-bound evidence must be stored byte-exact (no text normalization) and re-verified from a fresh clone

None of these should be collapsed into a generic "agent failed" state.

---

11 Future-proofing MCWS as context windows grow

The original R1.6.1 boundary remains appropriate.

MCWS should not be defended only as token reduction.

Its defensible runtime role is bounded capability exposure where smaller stage-specific sets improve operational clarity or selection behavior.

Provider independence, deterministic ranking, execution authority, mutation budgets and outcome validation belong to MCR, not MCWS.

If future models reason perfectly over arbitrarily large tool catalogs without selection/security cost, MCWS may become less important. The deterministic mission/binding/outcome architecture can remain useful independently.

---

12 What the experiment actually supports

The original benchmark supports the following bounded claim:

> **In the evaluated authored benchmark, stage-scoped MCWS achieved higher primary mission success than Broad and Static exposure.**

It does not establish:

- universal MCWS superiority;
- production latency savings;
- monetary savings;
- prompt-injection immunity;
- multi-tenant isolation;
- transaction correctness; or
- outcome correctness.

R1.7–R1.9 add runtime evidence, but they are not retroactively added as rows to the frozen MCWS experiment.

Further boundaries carried forward from the R1.6.1 claim-evidence audit:

- Broad versus Static is directional only (Holm-adjusted p = 0.2266).
- Mission-stage relevance as the specific cause of the MCWS advantage is **unresolved**: there is no size-matched selection-logic control, so smaller-versus-smarter scoping is not isolated. Broad also changes the model-planning mechanism and model-call count.
- Stage-visible exposure reduction is implementation-conformance evidence; downstream security, collision, latency and auditability benefits were not independently measured.
- The corrected primary analysis regraded persisted Static outputs against the already-frozen post-change common target (the M003 correction); no model, provider or SAP execution was rerun and no raw evidence was modified.
- The held-out split (Broad 0/12, Static 2/12, MCWS 7/12; n = 4 cases) is descriptive and directional only.

---

13 When to choose Static versus MCR

This section is a practitioner decision heuristic (architecture synthesis, evidence stream G). It is not an evidence-derived threshold and no study here measured the crossover.

Prefer **Static / compiled** when:

- one provider is stable;
- requirements are repetitive;
- operations are low-risk/read-only;
- execution is short-lived;
- deterministic throughput and simplicity dominate.

Prefer **MCR** when:

- provider availability/eligibility changes;
- requirements evolve mid-mission;
- tool estates overlap;

- delegated authority is material;
- consequential writes require bounded mutation authority;
- mission duration outlives one model session;
- point-of-use contract/policy changes are plausible; or
- authoritative outcome must be reconciled.

---

14 Platform map

The detailed vendor/platform map in R1.6.1 was a dated architecture map, not a vendor ranking.

This external-review candidate does not refresh that market table and performed no external market research. The R1.6.1 map is retained only as a dated 31 August 2026 snapshot. Every time-sensitive vendor, platform or specification statement in R1.6.1 is classified in the reference refresh register (`publication/MCR_R1.9_REFERENCE_REFRESH_REGISTER_R3.md`) and must be re-verified, or removed, before any public release.

No current-market claim should silently inherit freshness from the older whitepaper.

---

15 Build, buy and compose

The original practical rule still holds:

> **Buy the plumbing. Compose the providers. Build only the mission-semantic layer the enterprise uniquely needs.**

Buy/reuse:

- gateway;
- registry;
- OAuth/workload identity;
- API lifecycle;
- deterministic policy;
- observability.

Build narrowly:

- mission semantics;
- capability normalization;
- stage MCWS;
- mission-relative substitutability;
- deterministic binding state;
- transaction/mutation state;
- enterprise-specific outcome interpretation.

Compose:

- qualified MCP/API/event/enterprise providers.

---

16 Historical architecture ancestry: what is not new

MCR does not claim invention of:

- goal/capability mediation;
- semantic service descriptions;
- dynamic service binding;
- durable workflow state;
- Saga compensation;
- event-driven workflows; or
- tool retrieval.

Ancestry conceded by name: OWL-S and WSMO (semantic service description and goal-driven discovery; W3C member submissions, 2004–2005); SOA/BPM orchestration; Saga compensation; durable-execution systems such as Temporal; LLM tool retrieval and selection (ToolLLM, Gorilla, AnyTool, ToolRet, ToolScope, ToolChoiceConfusion, BiasBusters); and contemporaneous agent-governance work, including capability-contract and mission-bound-authorization proposals, agent identity protocols, and governed MCP orchestration (references [1]–[4], [10], [11], [13]–[15], [20], [22]–[24], [29]–[34] of R1.6.1). Several of these are emerging or vendor items and are time-sensitive; see the reference refresh register.

The residual research question is how these ideas should be recomposed for probabilistic agent reasoning when executable capability descriptions themselves become model input and when authority/outcome truth must remain deterministic.

---

17 Closest prior art and residual contribution

The residual contribution remains a differentiated combination rather than a claim of a wholly new computational primitive:

1. durable mission state independent of model/provider session;
2. mission-stage capability demand;
3. stage-scoped MCWS reasoning boundary;
4. semantic proposal separated from deterministic admissibility/binding;
5. point-of-use contract and authority revalidation;
6. explicit mutation/transaction state;
7. execution receipt separated from authoritative outcome;
8. replay/continuation semantics that do not assume side effects are absent after client failure.

Stated as capabilities: mission-scoped capability selection; deterministic execution authority; provider/tool binding; point-of-use revalidation; mutation budgets; transaction state; receipts; the outcome distinction; continuation and replay control; and enterprise-agent execution boundaries. Only the first is supported by a controlled comparison (the frozen MCWS benchmark); the others are supported at the evidence classes stated in Section 1 and Appendix E.

R1.9's strongest new evidence is not a clean success path. It is the explicit handling of ambiguous post-side-effect state.

---

18 Executive migration checklist

Before production adoption, require affirmative evidence for:

1. capability ownership/version traceability;
2. read/write/destructive semantics;
3. deterministic business authorization;
4. explainable provider eligibility;
5. directional substitutability;
6. correct resume identity/policy;
7. mutation budget;
8. replay barrier before first side effect;
9. contract drift handling;
10. authoritative readback independent of client success interpretation;
11. explicit partial-execution state; and
12. separate compensation authority.

---

19 Long-horizon thesis: from tool access to portable missions

The long-horizon architecture remains prerequisite-conditioned.

19.1 Directional substitution

R1.8 adds real contract-drift/requalification evidence but does not establish universal cross-provider equivalence.

19.2 Delegated authority

R1.9A formalizes delegation envelopes and attenuation rules. Full live multi-hop delegated mutation remains unestablished.

19.3 Transaction boundaries

R1.9B illustrates why invocation state and commit state must be distinct. Full multi-provider atomicity is not established.

19.4 Compensation

Compensation remained unauthorized in the frozen R1.9B live mission. No compensation-correctness claim is made.

19.5 Checkpointed continuation

MissionCheckpoint and ContinuationLease exist in the R1.9A foundation. Live continuation after an ambiguous committed side effect remains future work.

19.6 Contract evolution

R1.8 provides direct evidence that nominal provider version equality can coexist with contract fingerprint drift.

19.7 Autonomous self-healing

Not established.

Earlier live process-loss work observed fail-closed behavior and required explicit recovery. MCR must not claim autonomous ADT resurrection.

19.8 Multi-ERP federation

Deferred.

Federation, cross-organization trust, liability and economic routing remain beyond the R1.9 closure.

---

20 A buildable research program after R1.9

The next program should begin only after R1.9 closure is frozen.

Suggested independent POCs:

POC	Question	Evidence gate
N1	Can authoritative readback be independent of the mutation provider?	Side effect reconciled without trusting mutation response
N2	Can provider responses be normalized without losing contract identity?	Stable canonical outcome across qualified provider schemas
N3	Can a mission resume after a confirmed partial mutation without replay?	Checkpoint continuity + no duplicate side effect
N4	Can separately authorized compensation operate without authority amplification?	Explicit grant + bounded compensation evidence
N5	What is pure governance overhead under successful live mutation?	Separate business time, validation time and governance-only time
N6	Can delegated authority survive long-running resume/rebind?	No amplification; revocation observed
N7	Can multi-ERP mission state remain portable?	Bounded cross-system mission with independent outcome evidence

Each POC should receive its own prospective contract, authority boundary, freeze and evidence manifest.

---

21 Limitations and future work

The R1.9 program has several important limitations.

First, the original MCWS benchmark remains small and authored: 48 primary missions, four mission families and one local-model configuration. It does not establish population-level enterprise performance.

Second, R1.7/R1.8 provider/runtime evidence is bounded engineering evidence rather than a full production deployment.

Third, R1.9A is primarily deterministic/local foundation evidence.

Fourth, R1.9B did not reach validated live completion. CREATE reached the provider; ACTIVATE did not. The client then stopped because the CREATE response did not contain the expected file path, and authoritative SAP repository state could not be established before this draft. `CREATE_COMMIT_STATE=UNKNOWN` is therefore a scientific result, not a temporary wording inconvenience.

Fifth, no compensation was authorized or executed.

Sixth, full checkpointed continuation after a possibly committed side effect was not demonstrated.

Seventh, no autonomous self-healing claim is supported.

Eighth, multi-ERP federation, economic routing and cyber-physical execution remain long-horizon research.

Ninth, evidence portability is incomplete: the bytes of the frozen scientific manuscript (`E589393C...`) and the original signed bytes of six historical scripts are not in the repository (Appendix F). Claims about the MCWS figures rest on the R1.6.1 publication layer that reproduces them, not on recomputation from the frozen manuscript.

Tenth, the only overhead evidence (H38) is a local synthetic microbenchmark with identical percentile values; no end-to-end governance overhead under successful live mutation was measured.

The immediate publication task is therefore closure discipline, not another live write.

---

22 Conclusion

MCR began with a narrow post-LLM observation: when executable capability descriptions are placed inside model-active context, the tool catalog becomes part of probabilistic reasoning. The frozen MCWS benchmark supports stage-scoped capability exposure within its evaluated conditions, with MCWS achieving 30/48 primary mission successes compared with 17/48 for Static and 5/48 for Broad.

The broader Mission-Capability Runtime was originally presented as a practitioner pattern layered above protocols, gateways and systems of record. R1.7 and R1.8 added bounded engineering evidence for authority separation, provider contracts, point-of-use drift detection, multi-provider capability identity and governed continuation. R1.9A then formalized deterministic execution envelopes, mutation budgets, transaction state, contract leases, receipts and continuation objects.

R1.9B deliberately attempted the harder boundary: a prospectively frozen consequential SAP mutation. The program did not produce a clean end-to-end success. Instead, it exposed a sequence of fail-closed defects and ultimately reached the first mutation request and stopped when the CREATE response did not contain the expected file path. Because the CREATE call may have produced a side effect, replay was correctly blocked. Because ACTIVATE was not reached and authoritative repository readback remained unavailable, completion could not be claimed.

That result sharpens MCR's central thesis.

A robust enterprise-agent runtime must distinguish what the model proposed, what authority permitted, what runtime bound, what provider executed, what side effect committed and what enterprise state was later validated. These are not interchangeable milestones.

The strongest defensible R1.9 closure is therefore bounded:

> **The LLM proposes semantics; the runtime binds authority; enterprise state decides completion.**

MCR R1.9 should close with the partial/unknown live result preserved, rather than extending the experiment until a cleaner demo appears.

Appendix A — R1.9 closure classification

`MCR_R1_9=BOUNDED_CLOSURE_WITH_UNRESOLVED_CREATE_COMMIT_STATE`

No existing, already-captured authoritative read-only evidence establishes that CREATE persisted or did not. The alternative label, `BOUNDED_CLOSURE_WITH_PARTIAL_LIVE_MUTATION_RESULT`, requires authoritative proof of persistence and is therefore not used.

Interpretation: around a live enterprise mutation attempt, R1.9 exercised deterministic authority binding, MCP session binding, a frozen two-mutation budget, a single-use lock written before the first side effect, and a fail-closed stop with no retry. The lock's refusal of a replay was not itself exercised, and the budget was not consumed to completion. The frozen mission did not reach validated completion. CREATE was invoked against the ADT MCP provider; ACTIVATE was not; authoritative post-failure repository state remained unresolved. This supports bounded control-plane behavior. It does not establish complete live mutation/outcome correctness.

Appendix B — claims explicitly not made

This manuscript does not claim:

- autonomous self-healing;
- universal safe enterprise mutation;
- universal or general transaction atomicity;
- successful R1.9B end-to-end live completion;
- successful activation of `ZCL_MCR19B_9533C4D8`;
- proof that CREATE committed, or that it did not, without authoritative readback;
- compensation correctness;
- universal provider failover or equivalence;
- multi-ERP proof or federation safety;
- cyber-physical safety;
- end-to-end governance overhead under live mutation; or
- production certification.

Appendix C — relationship to ACLA and EAF

ACLA asks what the model should know. EAF asks whether the agent is authorized and sufficiently informed to act, and whether the outcome was validated. MCR asks which capability, provider or tool should satisfy the authorized mission under which deterministic execution controls. They are three programs, not one framework, and MCR does not replace ACLA or EAF.

```
Claude Mods -> ACLA -> EAF pre-action -> MCR -> EAF post-action -> ACLA / evidence
```

Appendix D — evidence streams

Stream	Content	Evidence class
A	Frozen R1.6 MCWS benchmark (Broad 5/48, Static 17/48, MCWS 30/48; held-out 0/12, 2/12, 7/12, n=4 descriptive)	Frozen experiment
B	Historical enterprise-engineering POCs (SAP, MCP, Capella, A4Layer, BAS/Fiori)	Engineering
C	R1.7 governed-runtime evidence (companion R2-E P3–P7, P6-D1)	Bounded experiments and live POCs

Stream	Content	Evidence class
D	R1.8 four-provider and multi-mission evidence	Bounded live engineering evidence
E	R1.9A governed-execution foundation	Synthetic / local
F	R1.9B bounded live mutation attempt	Partial live; commit state unknown
G	Architecture synthesis and exposition	Not empirical

Later streams are never added as rows to the frozen MCWS experiment.

Appendix E — hypothesis summary (R1.9)

H	Statement (short)	Final state
H31	Multi-MCP mission-scoped composition	OBSERVED (bounded; comparative reduction claim not established)
H32	Gateway + MCR temporal mission control	NOT_ESTABLISHED
H33	User -> Agent -> Mission attenuation	SYNTHETIC
H34	Outcome semantics over tool receipts	PARTIAL
H35	Mutation budgets; separately authorized compensation	PARTIAL (budget); NOT_ESTABLISHED (compensation)
H36	Contract fingerprinting and mission-aware drift	OBSERVED (change-classification part SYNTHETIC)
H37	Checkpointed resumption without replay	SYNTHETIC; live NOT_ESTABLISHED
H38	Bounded governance overhead	SYNTHETIC (local microbenchmark only)

Naming: the R1.8 companion paper labels these R2-H31 to R2-H38 and the R1.9A register carries H33–H38; this paper uses the short form. Full adjudication: `evidence/MCR_R1.9_FINAL_HYPOTHESIS_RESULT_MATRIX_R3.md`. R1.9C (live interruption and continuation, end-to-end overhead) remains deferred and is not initiated.

Appendix F — evidence portability gaps

- Frozen scientific manuscript `E589393C6EEA0BA92E4EC521D1577A6F561F9ADFA38FF9EEE97A94695C666D90`: hash cited by the R1.6.1 PDF and the RC1 package manifest; bytes not recovered.
- Six signed scripts (R1.8 watchdog, HF05 forensic, HF06 runner `DE183ABC...`, HF06 forensic, HF06 preflight reconciliation, HF06 adjudication script): original bytes unavailable; repository holds normalized reading copies that do not match the frozen hashes and must not be executed.
- The HF06 attempt-2 authorization is hash-only by design.
- R1.8 reviewer PDF package and the R1.7 P7 closure ZIP: bytes not in the repository (the register hash for the latter is malformed).
- Never retained: the raw HF06 CREATE response and the HF05 failure transcript. Claims that would need them are classified NOT_ESTABLISHED or OBSERVED-with-gap.
- None of these is claim-critical (see `reports/MCR_R1_9_PUBLICATION_EVIDENCE_GATE_R3.md`).

See `evidence/portable/MISSING_ARTIFACTS_R2.md` and `reports/MCR_R1_9_PUBLICATION_EVIDENCE_GATE_R3.md`.

Appendix G — work not done by this paper

- Independent external review of the claims (the claim audit and adversarial pass in the repository are self-reviews).
- Fresh external verification of time-sensitive and emerging references (none was performed).

- Recovery of the frozen scientific manuscript and the original signed bytes of six scripts, if they exist (Appendix F).
- Authoritative read-only repository readback of the R1.9B object, if the owner later authorizes it; this could refine the commit state to EXISTS or ABSENT without altering the rest of the record.
- R1.9C (live interruption and continuation; end-to-end governance overhead): deferred, not initiated.